

EXACT SYMBOLIC COMPUTATION

Functional Decomposition of Polynomials with Algebraic Coefficients

Characteristic-zero theory, exact algorithms,
and a Wolfram Language package

Prepared for Vladimir Reshetnikov

7 September 2026 | Version 1.0.0

Central result. Once the degree of an inner component is specified, its monic, zero-constant representative is uniquely determined by the leading coefficients of the input. It can be constructed without factoring a polynomial, solving a nonlinear system, choosing an analytic root branch, or extending the coefficient field.

This article supplies proofs of the reconstruction and descent theorems, a complete decision procedure, duplicate-free enumeration of complete decompositions, an audit of the derivative-factor method, exact algebraic arithmetic contracts, and a documented implementation.

Validation record. The independent exact reference implementation passed 574 checks. The package includes 107 native MUnit tests; they were not executed because the available Wolfram connector returned HTTP 404 and no local kernel was available. Native execution is distinguished from mathematical and reference-code validation throughout the article.

Abstract

Given $p \in \overline{\mathbb{Q}}[x]$, we seek a composition $p = f_1 \circ \cdots \circ f_r$ in which each nonlinear component is indecomposable. The coefficients may be represented by radicals, polynomial Root objects, or AlgebraicNumber values. We develop an exact, factorization-free procedure valid mathematically over every field of characteristic zero. For each proper divisor d of $n = \deg p$, put $m = n/d$. The unique possible normalized right component is the positive-degree part of the formal m th root of $p/\text{lc}(p)$ at infinity. A triangular recurrence constructs it, and an h -adic expansion decides whether it is actually a right component. This proves both completeness and descent to the original coefficient field. Recursive splitting yields a complete chain; enumeration by an indecomposable rightmost component gives all normalized complete chains without parenthesization duplicates. The accompanying Wolfram Language package uses exact coefficient vectors and coefficientwise RootReduce. Its elementary dense implementation takes $O(n^2\tau(n))$ coefficient-field operations for one complete decomposition, separately from the cost of algebraic-number arithmetic. Classical factorization-free decomposition and Ritt theory provide the historical and structural setting; the article derives the working algorithm directly.

Contents

1	The problem, and what the package promises	4
1.1	Composition, not multiplication	4
1.2	The mathematical domain and the implemented domain	4
2	Normalization and coefficient rigidity	4
2.1	Removing the affine ambiguity	5
2.2	The uniquely determined candidate	5
2.3	Descent: algebraic extensions do not help	6
3	Formal roots at infinity and an explicit recurrence	6
3.1	A root whose constant term is prescribed	6
3.2	Derivation of the recurrence	7
3.3	Relation to the usual approximate root	7
3.4	Why the recurrence is preferable to symbolic root expansion	8
4	The exact composition test: expansion in powers of the candidate	8
4.1	The h -adic expansion	8
4.2	A directly checkable decision record	9
4.3	A leading-term alternative	9
5	Decision, complete decomposition, and exhaustive enumeration	9
5.1	A complete fixed-degree decision procedure	9
5.2	One complete chain	10
5.3	Why arbitrary binary splitting duplicates answers	10

5.4	Finite output caps and honest completeness flags	11
5.5	Recovering nonnormalized decompositions	11
6	Structural consequences and the role of Ritt theory	11
6.1	What is invariant, and what is not	11
6.2	Explicit collisions	12
6.3	Monomials give exact enumeration benchmarks	12
6.4	Generic indecomposability and exactness	13
7	Exact algebraic-number arithmetic	13
7.1	Values, representations, and branch choices	13
7.2	Canonical reduction and exact zero testing	13
7.3	Input validation is a separate phase	14
7.4	Why there is no Extension option	14
8	Implementation architecture and complexity	15
8.1	Coefficient vectors and invariants	15
8.2	The candidate routine	15
8.3	Monic division and the membership routine	15
8.4	Cost of one trial	16
8.5	Cost of one complete recursive decomposition	16
8.6	Bit cost and coefficient-field growth	17
9	Worked examples	17
9.1	The radical-coefficient example from the question	17
9.2	The nested degree-24 example	18
9.3	A quintic Root coefficient	19
9.4	Mixed radicals and complex algebraic values	19
9.5	An indecomposable polynomial of composite degree	19
9.6	Two complete decompositions of a Chebyshev polynomial	20
10	Audit of the derivative-factor approach in the question	20
10.1	A single distinct derivative factor is not a rejection test	20
10.2	Subset enumeration duplicates divisors	21
10.3	The subtle issue of recursing only on the outer factor	21
10.4	The power-merging rewrite changes composition incorrectly	21
10.5	Exact algebraic identity checks should be explicit	21

11 Public Wolfram Language interface	22
11.1 Loading and evaluation conventions	22
11.2 One chain and one requested degree	22
11.3 All right components and all complete chains	22
11.4 The decision report	23
11.5 Composition and verification	23
11.6 Options and failures	23
12 Validation, reproducibility, and what remains untested	24
12.1 Executed exact reference checks	24
12.2 A useful warning from the external oracle	24
12.3 Prepared native tests, not a native test run	25
12.4 Reproducing the checks	25
13 Boundaries and further implementations	26
13.1 Positive characteristic is a genuinely different problem	26
13.2 Parameters and specialization	26
13.3 Faster arithmetic and candidate construction	27
13.4 Modular screening must respect specialization	27
13.5 Sparse inputs, rational functions, and approximate data	27
14 Conclusions	27
A Compact implementation map	28
B A small manual correctness checklist	28
C Archive layout and rebuilding the article	29
References	30

1 The problem, and what the package promises

1.1 Composition, not multiplication

The problem is to recognize and recover an identity

$$p(x) = f(h(x)), \quad \deg f > 1, \quad \deg h > 1. \quad (1)$$

It is unrelated to writing p as a product of irreducible polynomials. For example, $x^6 + x$ has a visible multiplicative factor x , but, as we prove below, it has no nontrivial functional decomposition. Conversely, compositional behavior cannot be read off just from the irreducible factors of p .

The motivating Mathematica Stack Exchange question asks whether the integer-coefficient behavior of `Decompose` can be extended to radicals. Its answer searches through divisors of p' and uses $(f \circ h)' = (f' \circ h)h'$. The supplied 2019 comment describes a historical implementation based on this idea [1]. We do not infer the behavior of a current Wolfram kernel from that historical statement. The package here has its own exact arithmetic and decomposition algorithm, rather than a wrapper that relies on a particular version of the built-in function. The official `Decompose` convention also puts the outermost polynomial first [4]; we use that order.

Factorization-free decomposition is classical, notably in the work of Kozen and Landau [2]. The aim here is not to claim a new decomposition paradigm, but to give a self-contained derivation, an exact algebraic-coefficient implementation, transparent failure behavior, and sufficient detail to audit or reimplement it.

1.2 The mathematical domain and the implemented domain

In the proofs, K is an arbitrary characteristic-zero field. In the package, K is the number field generated by the evaluated coefficients:

$$K = \mathbb{Q}(a_0, \dots, a_n) \subseteq \overline{\mathbb{Q}} \subset \mathbb{C}.$$

The implementation accepts exact algebraic numbers, real or complex. It does not accept unresolved coefficient parameters such as a in $x^4 + ax^2$, even though the mathematics works over $\mathbb{Q}(a)$. It does not attempt approximate decomposition of floating-point data.

The result of `AlgebraicDecompose[p, x]` is an outermost-first list $\{f_1, \dots, f_r\}$ whose composition equals p . When $\deg p \geq 2$, every f_i has degree at least two and is indecomposable. Every component except the first is monic and has constant coefficient zero. Constants and linear polynomials use the terminal convention $\{p\}$; they are not described as nonlinear indecomposables.

The other main operations are fixed-degree decomposition, all normalized right components, all complete normalized chains, an exact decision report, exact recomposition, and optional verification of completeness and normalization. Section 11 specifies these contracts precisely.

2 Normalization and coefficient rigidity

2.1 Removing the affine ambiguity

For any affine polynomial $\ell(y) = by + c$, $b \neq 0$,

$$f \circ h = (f \circ \ell) \circ (\ell^{-1} \circ h).$$

Consequently a request for literally every decomposition has infinitely many answers, even over $\mathbb{Q}$. A normalization is essential.

Definition 2.1. A right component h is *normalized* if $\text{lc}(h) = 1$ and $h(0) = 0$. A chain is normalized if all its components except possibly the first are normalized.

Suppose $p = f \circ g$, with $b = \text{lc}(g)$ and $c = g(0)$. Set

$$h(x) = \frac{g(x) - c}{b}, \quad F(y) = f(by + c). \quad (2)$$

Then $p = F \circ h$, and h is normalized. The outer polynomial is not in general monic: its leading coefficient must equal $\text{lc}(p)$.

2.2 The uniquely determined candidate

Write

$$p(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_0, \quad a_n \neq 0.$$

If $p = f \circ h$ with $\deg h = d$, then $n = md$, where $m = \deg f$. The possible nontrivial d are therefore precisely the proper divisors of n greater than one.

Theorem 2.2 (Fixed-degree rigidity). *Let $d \mid n$, $1 < d < n$, and $m = n/d$. There is at most one pair (f, h) satisfying $p = f \circ h$, $\deg h = d$, and h normalized. Moreover, the coefficients of its right component are determined by $a_n, a_{n-1}, \dots, a_{n-d+1}$ using only field arithmetic and division by nonzero integers.*

Proof. Write

$$h = x^d + b_1 x^{d-1} + \cdots + b_{d-1} x, \quad f = a_n y^m + u_{m-1} y^{m-1} + \cdots + u_0.$$

All terms of $f(h) - a_n h^m$ have degree at most $d(m-1) = n-d$. Hence, for $1 \leq k < d$, the coefficient of x^{n-k} in p must be that in $a_n h^m$. In expanding h^m , the term involving the new unknown b_k comes from selecting $b_k x^{d-k}$ in exactly one factor and the leading term in all the others. Any other contribution involves only $b_1, \dots, b_{k-1}$. Thus

$$\frac{a_{n-k}}{a_n} = m b_k + \Phi_k(b_1, \dots, b_{k-1}) \quad (3)$$

for a polynomial Φ_k with integer coefficients. Since $m \neq 0$ in K , these equations determine the b_k successively.

Once h is known, f is unique because substitution by a nonconstant polynomial is injective: for nonzero v , one has $\deg(v \circ h) = d \deg v$, so $v \circ h \neq 0$. $\square$

The theorem concerns *candidates*: solving the top-coefficient constraints does not prove that a decomposition exists. The lower coefficients provide an exact consistency test. This distinction is central to the implementation.

2.3 Descent: algebraic extensions do not help

Theorem 2.3 (Descent of normalized components). *Let $K \subseteq L$ be fields of characteristic zero and $p \in K[x]$. If p has a nontrivial decomposition in $L[x]$, its normalized right component and corresponding outer component belong to $K[x]$.*

Proof. Normalize the given decomposition in $L[x]$ by (2). Equations (3) then construct all coefficients of h from coefficients of p by arithmetic inside K . Thus $h \in K[x]$.

Recover f by repeatedly subtracting a scalar multiple of the appropriate power of h from p , starting at the leading term. Since h is monic, these scalars and every intermediate residual lie in $K[x]$. Equivalently, use the monic division algorithm proved in Section 4. It follows that all coefficients of f lie in K . $\square$

Corollary 2.4. *A polynomial in $K[x]$ is indecomposable over K if and only if it is indecomposable over any extension field of characteristic zero, including an algebraic closure of K .*

This is markedly different from multiplicative irreducibility. For functional decomposition, there is no need to guess which additional radicals to put into a factorization extension. The field generated by the input coefficients already contains the normalized answer. A result may display a different algebraic representation, but it represents an element of that same field.

Corollary 2.5 (Equivariance under embeddings). *Let $\sigma : K \hookrightarrow L$ be a field embedding. Whenever (f, h) is the normalized decomposition of p at right degree d , $(\sigma(f), \sigma(h))$ is the normalized decomposition of $\sigma(p)$ at degree d . A fixed degree has a decomposition before applying σ if and only if it has one afterwards.*

Proof. The candidate equations and all consistency equations use rational field operations. They commute with σ , which is injective and therefore preserves nonzero obstructions. $\square$

This explains why selected conjugate Root values can be processed uniformly. It does not license ignoring the selected conjugate: different embeddings generally give different numerical coefficients.

3 Formal roots at infinity and an explicit recurrence

3.1 A root whose constant term is prescribed

Put $t = 1/x$ and define

$$C(t) = \frac{t^n p(1/t)}{a_n} = 1 + c_1 t + c_2 t^2 + \cdots + c_n t^n, \quad c_j = \frac{a_{n-j}}{a_n}. \quad (4)$$

There is a unique formal power series

$$B(t) = 1 + b_1 t + b_2 t^2 + \cdots \in K[[t]] \quad \text{such that} \quad B(t)^m = C(t).$$

For example, its coefficients can be obtained recursively by comparing powers of t : the coefficient of the new b_k in B^m is m . The notation $C^{1/m}$ refers to this *formal* root with constant term one.

No analytic branch cut is involved. In particular, we never extract an m th root of a_n . Dividing by a_n first is what keeps the construction inside K , even when the leading coefficient is a complex algebraic number whose m th roots lie outside K .

Proposition 3.1 (Positive-degree part formula). *For a given $d \mid n$, $1 < d < n$, the unique possible normalized right component is*

$$h_d(x) = x^d + b_1x^{d-1} + \cdots + b_{d-1}x, \quad B = C^{1/m}, \quad m = n/d. \quad (5)$$

Only $C \bmod t^d$ is required.

Proof. The relation $B^m = C \pmod{t^d}$ gives exactly the top-coefficient conditions (3), after multiplying by x^n and putting $t = 1/x$. Uniqueness from Theorem 2.2 identifies the coefficients. $\square$

3.2 Derivation of the recurrence

Differentiating the formal identity $B^m = C$ gives

$$mCB' = C'B.$$

At coefficient t^{k-1} , the left side is $m \sum_{j=0}^{k-1} (k-j)c_j b_{k-j}$ and the right side is $\sum_{j=1}^k j c_j b_{k-j}$. Taking out the $j = 0$ term on the left yields

$$b_0 = 1, \quad b_k = \frac{1}{mk} \sum_{j=1}^k ((m+1)j - mk) c_j b_{k-j} \quad (1 \leq k < d). \quad (6)$$

Every term on the right involves an earlier b coefficient. The cost of computing the first $d - 1$ coefficients is $O(d^2)$ field operations. The first few are

$$b_1 = \frac{c_1}{m}, \quad (7)$$

$$b_2 = \frac{c_2}{m} - \frac{m-1}{2m^2} c_1^2, \quad (8)$$

$$b_3 = \frac{c_3}{m} - \frac{m-1}{m^2} c_1 c_2 + \frac{(m-1)(2m-1)}{6m^3} c_1^3. \quad (9)$$

These expressions are also obtained by the formal binomial series $(1 + U)^{1/m}$. The independent reference tests compare the recurrence with that separately implemented series operation.

3.3 Relation to the usual approximate root

In the Laurent series field $K((x^{-1}))$, set

$$S(x) = x^d B(x^{-1}) = (p(x)/a_n)^{1/m},$$

where the displayed equality has the formal meaning just specified. Let q be its full polynomial part, including the constant term. Then $S - q = O(x^{-1})$, and therefore

$$\deg(p/a_n - q^m) < n - d. \quad (10)$$

Indeed, $S^m - q^m = (S - q)(S^{m-1} + S^{m-2}q + \cdots + q^{m-1})$ has degree at most $-1 + d(m-1)$ at infinity. This q is the monic approximate m th root in the usual polynomial sense. Our candidate is $h_d = q - q(0)$, so computing $q(0)$ is unnecessary.

If $p = f(h)$ actually holds, expansion in the parameter $1/h$ gives

$$S = h + \frac{u_{m-1}}{ma_n} + O(h^{-1}) = h + \frac{u_{m-1}}{ma_n} + O(x^{-d}). \quad (11)$$

Thus subtracting the constant polynomial part is precisely the normalization $h(0) = 0$, rather than an arbitrary alteration of the candidate. The upper coefficients determine h , and the lower coefficients must make the formal root have the special shape in (11).

3.4 Why the recurrence is preferable to symbolic root expansion

One could try to ask a CAS to expand a radical expression near infinity. That brings together analytic branch conventions, possible sign cases, and simplification of symbolic limits, none of which is needed here. Equation (6) only performs exact operations on constants. It uses no `Series` call and no expression such as `PowerExpand[(p/a)^(1/m)]`. A formal construction with leading coefficient one is both more predictable and easier to verify.

4 The exact composition test: expansion in powers of the candidate

4.1 The h -adic expansion

Fix any monic polynomial h of degree $d \geq 1$. Repeated polynomial division produces

$$\begin{aligned} p &= q_1 h + r_0, \\ q_1 &= q_2 h + r_1, \\ &\vdots \\ q_s &= r_s, \end{aligned}$$

where $\deg r_j < d$ and the final quotient is zero after one more division. Consequently

$$p = r_0 + r_1 h + \cdots + r_s h^s, \quad \deg r_j < d. \quad (12)$$

These r_j are the polynomial analogues of digits in a positional numeral system.

Lemma 4.1 (Uniqueness of h -adic digits). *The representation (12) with $\deg r_j < d$ is unique.*

Proof. In a nonzero sum $\sum_j v_j h^j$ with $\deg v_j < d$, choose the largest j for which $v_j \neq 0$. Its degree lies in $[jd, (j+1)d - 1]$; every smaller-index term has degree below jd . Therefore its leading term cannot cancel. Applying this to the difference of two representations proves uniqueness. $\square$

Theorem 4.2 (Composition membership criterion). *For monic h , a polynomial p belongs to $K[h]$ if and only if every h -adic digit r_j is constant. In that case the outer component is*

$$f(y) = \sum_j r_j y^j.$$

Proof. Constant digits give the displayed composition immediately. Conversely, if $p = f(h) = \sum_j u_j h^j$ with $u_j \in K$, these constants already form a valid h -adic expansion. Lemma 4.1 forces $r_j = u_j$. $\square$

The first nonconstant remainder is therefore a conclusive obstruction, not merely a failed attempt to solve for an outer polynomial. Once the candidate degree is fixed, Theorem 2.2 excludes every alternative normalized right component of that degree.

4.2 A directly checkable decision record

For an unsuccessful degree d , the report stores h_d , the index j of the first nonconstant digit, that remainder, and the previous constant digits $u_0, \dots, u_{j-1}$. A checker can recover

$$q_j = \frac{p - \sum_{i < j} u_i h_d^i}{h_d^j}$$

by exact polynomial division and verify that the remainder of q_j modulo h_d is the reported nonconstant polynomial. It can also reconstruct h_d from the recurrence to check that the obstruction applies to the only normalized candidate.

For a successful degree, exact recomposition $f(h_d) - p = 0$ verifies the pair. To verify indecomposability of p , check a negative record for every proper degree divisor. The package's report is an exact decision record suitable for independent checking, but not a proof object already accepted by a proof-assistant kernel.

4.3 A leading-term alternative

An equivalent recovery algorithm subtracts multiples of h^j from the leading term downward. If a nonzero residual has degree not divisible by d , it cannot be a polynomial in h and the trial fails. Otherwise, subtract its leading coefficient times the corresponding power of h and continue. This also proves uniqueness of the outer polynomial.

Repeated monic division was chosen for the implementation because it never constructs all powers of h at once, has a small and explicit coefficient-domain interface, and directly returns the obstruction digits. It does not invoke Solve: the apparently nonlinear decomposition problem has become one deterministic candidate computation followed by polynomial division.

5 Decision, complete decomposition, and exhaustive enumeration

5.1 A complete fixed-degree decision procedure

For a specified degree d , the algorithm is:

1. Validate exact coefficients, compute the actual degree n , and require $1 < d < n$ with $d \mid n$.
2. Put $m = n/d$, form $c_1, \dots, c_{d-1}$, and compute $b_1, \dots, b_{d-1}$ using (6).
3. Form h_d as in (5), compute its h_d -adic digits, and reject at the first nonconstant digit.
4. If all digits are constant, form f from them and verify $f \circ h_d = p$ exactly before returning the pair.

Theorem 5.1 (Soundness, completeness, and termination). *The fixed-degree procedure terminates. It succeeds exactly when p has a right component of degree d , even if decompositions over extension fields are allowed. On success, it returns the unique normalized pair.*

Proof. The recurrence has $d - 1$ finite steps; each monic division and the sequence of divisions terminate by decreasing degree. On success, Theorem 4.2 proves equality. If a decomposition exists, Theorems 2.2 and 2.3 identify its normalized right component with the constructed candidate. Its digits are then constant by Theorem 4.2, so the procedure cannot reject. $\square$

Testing all proper divisors of n therefore decides whether p has any nontrivial decomposition. A polynomial of prime degree needs no trial. A polynomial of composite degree must still be tested: composite degree is necessary for decomposition, not sufficient.

5.2 One complete chain

To construct one chain, test the possible right degrees in a prescribed order. At the first success $p = f \circ h$, recursively decompose both f and h , then concatenate the two outermost-first lists. If every degree fails, return $\{p\}$.

Theorem 5.2 (Completeness of the recursive chain). *For $\deg p \geq 2$, this recursion terminates and returns a normalized complete chain. The result is valid for either increasing or decreasing trial-degree order.*

Proof. Both child degrees are strictly smaller than their parent's. Hence the recursion terminates. Inductively, concatenation preserves composition, and every returned leaf has no nontrivial right degree by Theorem 5.1.

Normalization also propagates. If the parent is monic and zero at zero, then in a split with normalized right component, the outer component is monic because leading coefficients multiply in the usual compositional way, and its value at zero is the parent's value at zero. Thus both children are normalized. Starting at the original input, only the outermost branch can retain a nonunit leading coefficient or a nonzero constant term. $\square$

Trying the smallest successful right degree also implies that the selected right component is indecomposable: a proper right component of it would give a smaller successful degree for p . Nevertheless, the implementation recursively processes both sides. Its correctness does not depend on that optimization or on a hidden property of trial order.

5.3 Why arbitrary binary splitting duplicates answers

Suppose $p = A \circ B \circ C$. A recursion that enumerates all binary splits can reach the same final chain from $(A \circ B) \circ C$ and from $A \circ (B \circ C)$. Exact algebraic equality makes post hoc expression deduplication more cumbersome than necessary. The duplication is caused by tree parenthesization, not by a genuine second decomposition.

The enumerator instead chooses the *rightmost indecomposable* component. For each normalized pair $p = f \circ h$, retain it only if h is indecomposable; enumerate complete chains of f and append h to each. If there are no nontrivial pairs, emit the singleton chain.

Theorem 5.3 (Enumeration without duplicates). *The preceding procedure enumerates exactly all normalized complete chains, with each affine-equivalence class represented once. No two outputs have the same ordered degree sequence.*

Proof. A complete chain has a rightmost indecomposable component h . At its degree, the fixed-degree test returns precisely that normalized h and its unique outer component f . The recursion on f supplies the prefix of the chain, by induction on degree. Thus every complete chain appears.

For uniqueness, the last degree determines h and f uniquely. The same argument applied to the prefix determines every earlier component. Induction therefore shows that a degree sequence determines at most one normalized chain and that no recursive branch duplicates another. $\square$

The code memoizes normalized right-pair lists locally during an `AlgebraicDecomposeAll` call. It does not install an unbounded global cache of user polynomials.

5.4 Finite output caps and honest completeness flags

A finite `MaxDecompositions` value M limits the returned list, not the mathematical search space. Emitting exactly M answers is not enough to know whether enumeration was exhaustive. The implementation searches until either it is exhausted or it has found $M + 1$ distinct chains. Only the latter case sets `EnumerationComplete` to `False` and discards the extra chain.

Consequently a polynomial with exactly M chains is correctly reported as fully enumerated with cap M . No count of omitted chains is claimed. The option is not a time or memory limit: candidate construction and memoized right-pair tests can require substantial work even before the first output is emitted.

5.5 Recovering nonnormalized decompositions

For a normalized chain $F_1, \dots, F_r$, choose affine polynomials $\ell_1, \dots, \ell_{r-1}$. Every chain in its affine-equivalence class has the form

$$\begin{aligned} &F_1 \circ \ell_1, \\ &\ell_1^{-1} \circ F_2 \circ \ell_2, \\ &\quad \dots, \\ &\ell_{r-2}^{-1} \circ F_{r-1} \circ \ell_{r-1}, \\ &\ell_{r-1}^{-1} \circ F_r. \end{aligned}$$

The adjacent affine maps cancel in composition. These infinitely many choices are deliberately not enumerated. The returned normalized chains are a finite, unambiguous set of representatives.

6 Structural consequences and the role of Ritt theory

6.1 What is invariant, and what is not

Ritt's first theorem connects complete decompositions by elementary changes involving adjacent factors. In characteristic zero, all complete chains of the same polynomial have the same length and the same *multiset* of component degrees, though their degree order can differ. A modern treatment gives the degree statement as Corollary 2.12 [3]. Descent allows the degree invariant to be applied to the coefficient field used here, not just to $\mathbb{C}$.

This structural theorem is not used as a substitute for an algorithmic proof. Theorems 5.1–5.3 establish the decision and enumeration procedures directly.

Corollary 6.1 (Minimizing the largest component degree). *Among all decompositions of a fixed nonlinear polynomial into nonlinear polynomial components, every complete decomposition attains the least possible value of the largest component degree.*

Proof. Refining a component of degree ab , with $a, b > 1$, replaces it by components of degrees a and b , both smaller than ab . Thus refinement to a complete chain cannot increase the maximum degree. Ritt's degree invariant makes that final maximum independent of the selected complete chain. No original chain can have a smaller maximum than its own complete refinement. $\square$

This does not say that all indecomposable components have prime degree. For the nested example in Section 9.2, the invariant multiset is $\{2, 3, 4\}$; the indecomposable quartic cannot be replaced by quadratics in another complete chain.

6.2 Explicit collisions

Two useful families of identities can be checked directly:

$$T_m \circ T_n = T_n \circ T_m, \quad (13)$$

$$x^n \circ (x^s R(x^n)) = (x^s R(x)^n) \circ x^n. \quad (14)$$

Here T_k is the Chebyshev polynomial. The first identity follows by putting $x = \cos \theta$, using $T_k(\cos \theta) = \cos(k\theta)$, and then using equality of polynomials on infinitely many points. Both sides of the second identity equal $x^{ns} R(x^n)^n$.

Under the appropriate coprime degree conditions, these Chebyshev and power-type identities, together with affine changes, describe the nontrivial collisions in Ritt's second theorem [3, Theorem 2.17]. The package need not recognize these special forms: it finds the corresponding chains by the same fixed-degree tests as for a generic polynomial.

6.3 Monomials give exact enumeration benchmarks

For $p = x^N$, every candidate is $h_d = x^d$, and every proper divisor d succeeds with outer polynomial $x^{N/d}$. A complete chain consists precisely of monomials whose degrees are the prime factors of N , in some order. If

$$N = \prod_{i=1}^s p_i^{e_i}, \quad r = \sum_i e_i,$$

then the number of normalized complete chains is

$$\frac{r!}{e_1! \cdots e_s!}. \quad (15)$$

This follows by counting distinct permutations of the degree multiset; Theorem 5.3 proves that each sequence determines one chain. These counts give a test oracle independent of a CAS decomposition routine. For $N = 6, 12, 16, 30, 36, 72$, the counts are respectively 2, 3, 1, 6, 6, 10.

6.4 Generic indecomposability and exactness

For a fixed n and a fixed proper divisor d , the coefficients of h_d are rational functions of $a_0, \dots, a_n$ with denominators nonzero on the chart $a_n \neq 0$. The h_d -adic remainders have the same property. The condition that their nonconstant coefficients vanish can therefore be expressed by finitely many polynomial equations after clearing those denominators.

Thus decomposability at a fixed degree is a Zariski-closed condition inside the degree- n coefficient chart. Taking the finite union over d preserves closedness. This union is proper: $x^n + x$ lies outside it for every $n \geq 2$. For a nontrivial d , its candidate is x^d and the first remainder is x , which is nonconstant; if there is no such d , indecomposability follows from prime degree.

It follows directly that indecomposability is a nonempty Zariski-open condition in that chart. Exact compositional identities can disappear under arbitrarily small generic perturbations. This is why replacing exact algebraic coefficients by decimal approximations changes the problem, rather than merely slowing the same decision procedure.

7 Exact algebraic-number arithmetic

7.1 Values, representations, and branch choices

A coefficient such as $\sqrt{2}$, a selected root of $z^5 - z - 1$, or $\sqrt{2} + i$ denotes an exact number. The package must preserve that number, not just its minimal polynomial. Distinct conjugates satisfy the same minimal polynomial but can give distinct input polynomials.

Radicals have their usual Wolfram Language value semantics. For example, a principal complex cube root is not silently replaced by a real cube root. The implementation never calls `PowerExpand` and never rewrites radicals by identities that are valid only under unstated sign assumptions.

The package is concerned with *constant algebraic* Root objects. A Root expression depending on the polynomial variable is not an algebraic coefficient. A numerical or transcendental root is also outside the domain. Membership in `Algebraics`, together with the representation checks described below, is required [7].

7.2 Canonical reduction and exact zero testing

The arithmetic layer applies `RootReduce` separately to coefficients, not to an entire polynomial in the indeterminate x . Wolfram documents `RootReduce` as reducing algebraic combinations of exact algebraic numbers to a canonical algebraic representation; it can simplify a result to a rational number or a simpler form where appropriate [5].

An integer or rational coefficient takes a fast path and is left alone. Other supported coefficients are reduced exactly. Equality testing is then implemented by reducing a difference and comparing it with the exact integer zero. No tolerance, `Chop`, `PossibleZeroQ`, numerical sampling, or floating-point norm is used.

It is not necessary for two equal input coefficients to have literally identical printed forms. What matters is that their reduced difference is exactly zero. Leading cancellations must be resolved before computing the degree. For example, if $\alpha^5 - \alpha - 1 = 0$, a displayed term $(\alpha^5 - \alpha - 1)x^{20}$ contributes nothing to the actual degree. The package reduces the coefficient vector and trims exact leading zeros.

7.3 Input validation is a separate phase

The input expression is expanded and checked with `PolynomialQ` in an unassigned, nonnumeric symbol. Its coefficient list is then obtained. Any surviving machine-precision or arbitrary-precision Real coefficient is rejected, before any attempt to reinterpret it as an exact number.

Each reduced coefficient must pass a closed algebraic-expression grammar: integers, rationals, exact complex numbers with algebraic parts, suitable `Root` or `AlgebraicNumber` values, and algebraic sums, products, or rational powers. Numeric status and exact algebraic-domain membership are checked as well. A coefficient that cannot be certified in this supported representation is rejected with a `Failure` object.

This is intentionally stricter than the abstract characteristic-zero algorithm. For example, the theory permits a rational function field, but this implementation does not maintain assumptions about a symbolic parameter or split into specialization cases. A failure to certify an input is not reported as mathematical indecomposability.

As usual for an ordinary Wolfram function without holding attributes, arguments evaluate before the function receives them. These checks apply to the evaluated coefficient values, not to the historical origin of an expression before evaluation. The variable should be cleared before calling the API.

7.4 Why there is no Extension option

Factoring a derivative needs a chosen coefficient field in which to factor. The present procedure only needs field arithmetic, so the normalized answer is already in the field generated by the input. Theorem 2.3 explains why an extension parameter would not add missing decompositions in characteristic zero.

The implementation does not explicitly construct one common primitive element. `RootReduce` handles exact intermediate coefficient values. This favors a short, auditable arithmetic boundary, at the cost of potentially repeated algebraic-number conversion work.

An alternative backend can choose θ with $K = \mathbb{Q}(\theta)$ and a minimal polynomial M , and represent every coefficient as a residue class in $\mathbb{Q}[z]/(M)$. Addition is vector addition; multiplication is polynomial multiplication modulo M ; a nonzero inverse is obtained by the extended Euclidean algorithm. The chosen complex embedding of θ must be retained when converting results back to Wolfram expressions. The arithmetic itself then takes place in a single exact quotient field.

The official `ToNumberField` documentation provides common-field conversion for lists. It distinguishes `Automatic`, which need not find the smallest field, from the `All` choice that asks for the smallest common field [6]. Such a backend would be a performance extension to this release, not a prerequisite for its mathematical completeness. No explicit primitive-element backend is claimed to be implemented here.

8 Implementation architecture and complexity

8.1 Coefficient vectors and invariants

Internally a polynomial is an ascending coefficient vector:

$$[a_0, a_1, \dots, a_n] \longleftrightarrow \sum_{i=0}^n a_i x^i.$$

The zero polynomial is exactly $\{0\}$, not an empty list. Vectors have no trailing zero entries except this zero representation. A normalized degree- d right component has first entry zero and last entry one. Every arithmetic result is reduced coefficientwise and trimmed.

The trusted arithmetic boundary consists of exact coefficient reduction and ordinary exact scalar arithmetic. The package itself implements dense vector addition, multiplication, composition by Horner evaluation, and monic polynomial division. It does not delegate the decomposition to `Decompose`, factorization to `Factor`, or coefficient recovery to `Solve`.

The source is organized in four layers: input and arithmetic checks; coefficient-vector operations; candidate construction and h -adic recognition; and public decision, recursion, enumeration, and reporting functions. Failures use a private tagged `Catch/Throw` channel, so an invalid coefficient or arithmetic problem is not confused with a negative decomposition result.

8.2 The candidate routine

The following is the actual core from the supplied package. In this listing, `red` is the exact coefficient reducer and vectors are ascending.

```
rightCandidate[p_List, d_Integer] := Module[
  {n = Length[p] - 1, m, c, b, k, j},
  m = Quotient[n, d];
  c = Table[red[p[[n - j + 1]]/Last[p]],
    {j, 0, d - 1}];
  b = ConstantArray[0, d]; b[[1]] = 1;
  Do[
    b[[k + 1]] = red[
      Total[Table[
        ((m + 1) j - m k) c[[j + 1]] b[[k - j + 1]],
        {j, 1, k}]]/(m k)],
    {k, 1, d - 1}];
  Prepend[Reverse[b], 0]
];
```

The reversal converts $[1, b_1, \dots, b_{d-1}]$ to the ascending coefficients of the positive-degree terms; prepending zero imposes the constant normalization. All calls to this internal routine are guarded by the proper-divisor test.

8.3 Monic division and the membership routine

For a monic divisor h of degree d , descending long division reads the current leading coefficient s , writes it into the quotient, and subtracts $sx^{k-d}h$. The leading coefficient of h is one, so no additional

inversion is required. Each changed coefficient is reduced immediately, preserving exact zero and degree invariants.

The membership routine stops at its first nonconstant digit:

```
outerData[p_List, h_List] := Module[
  {q = p, digits = {}, qr, j = 0},
  While[Length[q] > 1,
    qr = vDivideMonic[q, h];
    If[Length[qr[[2]]] > 1,
      Return[<|"Success" -> False, "DigitIndex" -> j,
        "RemainderVector" -> qr[[2]],
        "PriorDigits" -> digits|>]];
    AppendTo[digits, First[qr[[2]]]];
    q = qr[[1]]; j++;
  AppendTo[digits, First[q]];
  <|"Success" -> True, "OuterVector" -> trim[digits]|>
];
```

A vector of length one represents a constant, including zero. Successful pairs are recomposed exactly by the caller. Complete chains and enumerated chains are also recomposed before being returned. These redundant identity checks can expose arithmetic or indexing defects, though recomposition alone does not prove completeness of a decomposition.

8.4 Cost of one trial

Count operations in K , treating each exact coefficient operation as one unit. Candidate construction takes $O(d^2)$ such operations. A monic division of a degree- N polynomial by a degree- d polynomial costs $O((N - d + 1)d)$. In the h -adic expansion, the quotient degrees decrease by d , so the total is bounded by

$$O\left(d \sum_{j=1}^m (n - jd + 1)\right) = O(n^2).$$

Horner recomposition of an outer polynomial of degree m and an inner polynomial of degree d has the same $O(n^2)$ dense bound: its successive multiplications have costs $O(jd^2)$, and $\sum_{j \leq m} jd^2 = O(m^2 d^2)$.

There are at most $\tau(n) - 2$ proper degree divisors, where τ is the divisor-counting function. Testing every one, or stopping at a first success, therefore costs at most $O(n^2 \tau(n))$ field operations. Early rejection often makes individual unsuccessful trials cheaper.

8.5 Cost of one complete recursive decomposition

At a split of degree $N = ab$ with $a, b \geq 2$,

$$a^2 + b^2 \leq \frac{N^2}{2}.$$

For example, dividing by $a^2 b^2$ reduces this to $1/a^2 + 1/b^2 \leq 1/2$. Thus the sum of squared node degrees at successive levels of the recursion tree decreases geometrically. The sum over all nodes is at most $2n^2$. Every node degree divides n , so its divisor count is at most $\tau(n)$.

It follows that the entire one-chain decomposition still has the field operation bound

$$O(n^2\tau(n)), \quad (16)$$

including the final dense recomposition. A trial needs $O(n)$ coefficient slots apart from the representation sizes of coefficients; retaining all reports or memoized enumeration states needs more storage.

This counts coefficient-field operations, not expression parsing or integer-divisor generation. It is an elementary bound for this implementation, not a claim of optimal polynomial-decomposition complexity. The classical literature contains more sophisticated algorithms [2]. The supplied code favors transparent exact operations over asymptotic optimization.

8.6 Bit cost and coefficient-field growth

A field-operation count is not a bit-complexity bound. Let $D = [K : \mathbb{Q}]$. Exact arithmetic in K depends on D , rational coefficient heights, the chosen representation, and the sizes of intermediate numerators and denominators. Independent radicals can generate a field whose degree grows rapidly with the number of generators. Canonical RootReduce conversions can dominate the polynomial-vector work.

The implementation is dense. A sparse expression such as $x^{1000000}$ is expanded into a coefficient list of length 1000001; no sparse complexity claim is made. The enumeration cost additionally depends on the number of chains and the memoized states explored. A finite output cap is not an asymptotic bound on that search.

9 Worked examples

The expressions in this section are exact mathematical results, checked by the independent reference implementation. Wolfram snippets specify usage and expected values, not a transcript of an executed native kernel.

9.1 The radical-coefficient example from the question

Put $a = \sqrt{2}$ and

$$\begin{aligned} p(x) = & 3 + 3a + (14 + 4a)x + (12 + 26a)x^2 \\ & + (56 + 8a)x^3 + (8 + 48a)x^4 + 48x^5 + 16ax^6. \end{aligned}$$

The only nontrivial right degrees to test are two and three. For $d = 2$, $m = 3$ and

$$c_1 = \frac{48}{16\sqrt{2}} = \frac{3}{\sqrt{2}}, \quad b_1 = \frac{c_1}{3} = \frac{1}{\sqrt{2}}.$$

Thus the unique candidate is

$$h(x) = x^2 + \frac{x}{\sqrt{2}}.$$

Repeated division gives constant digits, and the outer polynomial is

$$F(y) = 3 + 3\sqrt{2} + (8 + 14\sqrt{2})y + (8 + 24\sqrt{2})y^2 + 16\sqrt{2}y^3. \quad (17)$$

Both component degrees are prime, so the pair is complete.

The original normalization in the question uses $g = \sqrt{2}h = x + \sqrt{2}x^2$. Its outer component is $F(y/\sqrt{2})$, namely

$$3 + 3\sqrt{2} + (14 + 4\sqrt{2})y + (4 + 12\sqrt{2})y^2 + 8y^3.$$

The package's different printed form is therefore the same decomposition with a canonical inner normalization.

For $d = 3, m = 2$, the candidate is

$$h_3 = x^3 + \frac{3\sqrt{2}}{4}x^2 + \left(\frac{15}{16} + \frac{\sqrt{2}}{8}\right)x.$$

Already the first remainder of p modulo h_3 is

$$\left(\frac{3}{2} + \frac{51\sqrt{2}}{16}\right)x^2 + \left(\frac{51}{16} + \frac{3\sqrt{2}}{4}\right)x + 3 + 3\sqrt{2}. \quad (18)$$

It is nonconstant, so degree three is conclusively excluded. There is exactly one normalized complete chain for this example.

```
Clear[x];
p = 3 + 3 Sqrt[2] + (14 + 4 Sqrt[2]) x +
    (12 + 26 Sqrt[2]) x^2 + (56 + 8 Sqrt[2]) x^3 +
    (8 + 48 Sqrt[2]) x^4 + 48 x^5 + 16 Sqrt[2] x^6;
chain = AlgebraicDecompose[p, x];
AlgebraicVerifyDecomposition[p, chain, x,
    "RequireComplete" -> True, "RequireNormalized" -> True]
(* Expected: True *)
AlgebraicDecomposeAtDegree[p, x, 3]
(* Expected: Missing["NotDecomposableAtDegree", 3] *)
```

9.2 The nested degree-24 example

Set $p_{24}(x) = p(x^4 - x + 1)$. A normalized complete chain is

$$\begin{aligned} A(y) &= 141 + 105\sqrt{2} + (168 + 142\sqrt{2})y \\ &\quad + (56 + 72\sqrt{2})y^2 + 16\sqrt{2}y^3, \\ B(y) &= y^2 + \left(2 + \frac{1}{\sqrt{2}}\right)y, \\ C(x) &= x^4 - x. \end{aligned}$$

Thus $p_{24} = A \circ B \circ C$ and the degree sequence is $(3, 2, 4)$. One can derive the coefficients without expanding a degree-24 polynomial: write $x^4 - x + 1 = C + 1$, expand $h(C + 1)$ as a quadratic in C , move its constant into the cubic outer component, and use (17).

The quartic C is indecomposable. Its only possible nontrivial inner degree is two; the top coefficients give candidate x^2 , and its first remainder is $-x$. Both ascending and descending trial orders must still split any decomposable factor produced earlier. Processing both recursive children makes this independent of the initially selected degree.

```

p24 = Expand[p /. x -> x^4 - x + 1];
c24 = AlgebraicDecompose[p24, x];
Exponent[#, x] & /@ c24
(* Expected: {3, 2, 4} *)
AlgebraicVerifyDecomposition[p24, c24, x,
  "RequireComplete" -> True, "RequireNormalized" -> True]
(* Expected: True *)

```

9.3 A quintic Root coefficient

Let α be the first Wolfram polynomial root of $z^5 - z - 1$ and put

$$h = x^2 + \alpha x, \quad f = y^3 + \alpha y + 1, \quad p = f(h).$$

The exact normalized complete chain is (f, h) . The algorithm uses $\alpha^5 = \alpha + 1$ in exact coefficient arithmetic and does not need a radical expression for α .

```

alpha = Root[#^5 - # - 1 &, 1];
pRoot = Expand[(x^2 + alpha x)^3 +
  alpha (x^2 + alpha x) + 1];
AlgebraicDecompose[pRoot, x]
(* Expected exact values:
  {x^3 + alpha x + 1, x^2 + alpha x} *)

```

The reference tests also use randomized decompositions in this quintic field. The prepared native suite includes a selected complex conjugate as well as the first root, so that value selection is not replaced by a minimal-polynomial-only comparison.

9.4 Mixed radicals and complex algebraic values

In $K = \mathbb{Q}(\sqrt{2}, \sqrt{3}, i)$, take

$$h = x^2 + (\sqrt{2} + i)x, \quad f = y^3 + \sqrt{3}y + 1.$$

The field has degree eight over $\mathbb{Q}$; the reference tests represent it as an exact algebraic field and recover these prescribed components from their expanded composition. There is no special real-coefficient branch in the decomposition procedure. The complex coefficient is simply an exact element of K .

9.5 An indecomposable polynomial of composite degree

For $p = x^6 + x$, the degree-two candidate is x^2 , and the degree-three candidate is x^3 . Each first remainder is x . Therefore neither candidate succeeds, and the complete chain is the singleton $\{x^6 + x\}$. This example tests the difference between a prime-degree shortcut and an actual negative decision at composite degree.

A report contains the two obstructions:

```
report = AlgebraicDecompositionReport[x^6 + x, x];
report["Status"]
(* Expected: "Indecomposable" *)
report["DecisionComplete"]
(* Expected: True *)
```

9.6 Two complete decompositions of a Chebyshev polynomial

For

$$T_6(x) = 32x^6 - 48x^4 + 18x^2 - 1,$$

the two normalized complete chains are

$$(32y^3 - 48y^2 + 18y - 1) \circ x^2,$$

$$(32y^2 - 1) \circ \left(x^3 - \frac{3}{4}x\right).$$

Their ordered degrees are (3, 2) and (2, 3). Direct substitution verifies both identities. Since two and three are the only proper degree divisors, fixed-degree uniqueness also proves that this list is complete.

```
all = AlgebraicDecomposeAll[ChebyshevT[6, x], x];
all["ReturnedCount"]
(* Expected: 2 *)
all["EnumerationComplete"]
(* Expected: True *)
AlgebraicDecomposeAll[x^12, x, "MaxDecompositions" -> 2]
(* Three chains exist; returns two with EnumerationComplete -> False. *)
AlgebraicDecomposeAll[x^12, x, "MaxDecompositions" -> 3]
(* Returns all three with EnumerationComplete -> True. *)
```

10 Audit of the derivative-factor approach in the question

The chain-rule method is mathematically legitimate: if $p = f \circ g$, then g' divides p' . In characteristic zero, integrating a nonzero scalar multiple of g' recovers g up to precisely the affine freedom that normalization removes. An exhaustive search through divisors, followed by an exact composition test, can therefore work. The difficulty is not the existence of an algorithm; it is the avoidable combinatorial search and some correctness details in the supplied implementation.

10.1 A single distinct derivative factor is not a rejection test

The code returns immediately when the nonconstant part of `FactorList` has fewer than two entries. Factor-list entries count *distinct* factors, not their multiplicities. For example,

$$p = x^4, \quad p' = 4x^3, \quad p = x^2 \circ x^2.$$

There is only one distinct nonconstant derivative factor, but a valid nontrivial decomposition exists. The early return misses it. Multiplicity must be considered if this method is retained.

10.2 Subset enumeration duplicates divisors

Suppose the factorization is $p' = c \prod_i q_i^{e_i}$. The multiset of individual factor copies has $E = \sum_i e_i$ members and 2^E subsets. Many subsets give the same divisor when multiplicities occur. The number of distinct monic divisors is only $\prod_i (e_i + 1)$. Exponent-vector enumeration avoids those duplicates, but the resulting search can still be very large. In contrast, the factorization-free method has only one candidate for each proper degree divisor of n .

The derivative degree filters should also use the exact constraints

$$d = \deg g = 1 + \deg g', \quad d \mid n, \quad 2 \leq d \leq n/2.$$

In particular, the upper bound for $\deg g'$ is $n/2 - 1$, not $n/2$. Constructing an outer polynomial before checking $d \mid n$ leads to needless or malformed candidate systems based on the noninteger quotient n/d .

10.3 The subtle issue of recursing only on the outer factor

It would be too strong to say that outer-only recursion is always wrong for the exact exhaustive subset order in the supplied code. Suppose the selected g were itself decomposable, $g = A \circ B$. Then $g' = (A' \circ B)B'$, and B' is a proper divisor of g' with fewer irreducible factor copies. Its integrated candidate also gives a right component of p . If every subset is tested in increasing cardinality with a complete exact composition test, the candidate from B' would have succeeded earlier. Under those assumptions, the first successful g is indecomposable.

That is an *order-dependent proof obligation*, not an automatic property of every derivative search. A refactoring to degree heuristics, a different divisor generator, or an early search limit can invalidate it. The new package recursively decomposes both components explicitly, so it requires no such implicit guarantee. Ascending degree order gives an additional simple justification, but descending order remains correct.

10.4 The power-merging rewrite changes composition incorrectly

The supplied rewrite replaces adjacent x^n and x^m by x^{n+m} . For composition, however,

$$x^n \circ x^m = x^{nm}, \quad \text{not } x^{n+m}.$$

Thus whenever the rule applies with $nm \neq n + m$, it changes the represented polynomial. Replacing addition by multiplication restores composition equality, but still merges factors and can destroy a *complete* decomposition. A complete-chain API should not merge adjacent prime-degree powers at all.

10.5 Exact algebraic identity checks should be explicit

A generic symbolic zero heuristic is unnecessary for algebraic numbers. Reduce each coefficient exactly and check equality to zero. The new package also validates the coefficient domain before starting, so an unsupported parameter, an inexact input, and a proved negative decomposition result have different return values.

The derivative-factor strategy can be repaired, but no repair is needed for the new core: it never factors p' and never enumerates divisors of that derivative. The only divisors it enumerates are integer divisors of $\deg p$.

11 Public Wolfram Language interface

11.1 Loading and evaluation conventions

The simplest installation is to load the package file directly:

```
base = "C:/path/to/AlgebraicDecomposition";
Get[FileNameJoin[{base, "Kernel", "AlgebraicDecomposition.wl"}]];
Clear[x];
```

The package exports symbols in the context `AlgebraicDecomposition``. It does not unprotect, replace, or add definitions to the system `Decompose`. The supplied `Kernel/init.m` and `PacletInfo.wl` also describe an optional package/paclet layout. No paclet installation is required for direct `Get`.

The intended platform is Wolfram Language 13.0 or later. That is a compatibility target, not a measured minimum: no native kernel execution was available for this release. Run the supplied `MUnit` suite in the target kernel before relying on native compatibility.

11.2 One chain and one requested degree

```
AlgebraicDecompose[p, x, "DegreeOrder" -> "Ascending"]
AlgebraicDecompose[p, x, "DegreeOrder" -> "Descending"]
AlgebraicDecomposeAtDegree[p, x, d]
```

The first two calls select the order of tested right degrees and return one complete normalized chain. Different orders may return different valid chains; they need not do so for every polynomial.

A successful fixed-degree call returns $\{f, h\}$. When d is a valid proper divisor but its unique candidate fails, the result is

```
Missing["NotDecomposableAtDegree", d]
```

An invalid requested degree, such as $d = 1$, $d = n$, or $d \nmid n$, returns a `Failure`, not `Missing`. This distinction separates an invalid question from a mathematically negative answer to a valid one.

11.3 All right components and all complete chains

```
AlgebraicRightComponents[p, x]
AlgebraicDecomposeAll[p, x, "MaxDecompositions" -> Infinity]
```

The right-component call returns all nontrivial normalized pairs $\{f, h\}$, in increasing order of $\deg h$. It returns an empty list when there is no such pair. These are two-component decompositions, not necessarily complete chains; either component may itself decompose.

The all-chain call returns an association:

```
<|"Decompositions" -> {chain1, chain2, ...},
  "EnumerationComplete" -> True,
  "ReturnedCount" -> numberOfReturnedChains|>
```

The cap must be Infinity or a positive integer. When truncation is detected, EnumerationComplete is False. The reported count is only the number of returned chains. For a constant, linear, or indecomposable input, there is one terminal chain, the singleton $\{p\}$, under the API convention.

11.4 The decision report

```
AlgebraicDecompositionReport[p, x]
```

The top-level association has the keys Polynomial, Degree, Status, DecisionComplete, and Trials. Status is one of Constant, Linear, Decomposable, or Indecomposable. The zero polynomial is reported with degree -Infinity. A normally returned report tests every proper divisor, so DecisionComplete is True; an arithmetic failure produces a Failure instead of a misleading incomplete negative report.

Each trial contains the right degree, the outer degree, the normalized candidate, and DecomposableAtDegree. A success adds OuterComponent. A failure adds ObstructionDigit, NonconstantRemainder, and PreviousConstantDigits. Digit indices start at zero, as in (12).

11.5 Composition and verification

```
AlgebraicCompose[chain, x]
AlgebraicVerifyDecomposition[p, chain, x,
  "RequireComplete" -> False,
  "RequireNormalized" -> False]
```

Composition uses the outermost-first convention. In particular, AlgebraicCompose[{f,g,h},x] computes $f(g(h(x)))$. The empty chain represents the identity x .

Verification always checks exact equality to p . With RequireNormalized set to True, all components after the first must be monic, zero at zero, and nonconstant. With RequireComplete set to True for a nonlinear input, every component must have degree at least two and have no nontrivial right component. For a constant or linear input, completeness means a singleton chain. Consequently the empty chain verifies the identity as a composition but is not called a complete decomposition by this convention.

An equality or structural mismatch returns False. An invalid coefficient domain returns Failure, not False. Completeness verification repeats exact decomposition tests on the components, so it can cost substantially more than an identity-only check.

11.6 Options and failures

Options in this release use string names. Unknown options and values outside the documented choices produce Failure objects. The principal tags are summarized below.

Tag	Meaning
InvalidArguments	Wrong arity or argument kind.
InvalidVariable	A numeric symbol was supplied as the variable.
NotPolynomial	The evaluated expression is not polynomial in the variable.
InexactCoefficient	An inexact real value remains in the coefficient list.
NonAlgebraicCoefficient	Exact algebraic membership could not be certified.
InvalidRightDegree	The requested degree is not a proper divisor greater than one.
UnknownOption, InvalidOption	Unsupported option name or value.
AlgebraicArithmetic	Exact coefficient reduction did not complete normally.
InternalInvariant, InternalVerification	A required internal property or exact recomposition check failed.

An unsupported computation is never intentionally converted to the singleton $\{p\}$, because that would falsely suggest indecomposability. The package has no probabilistic success mode or hidden numerical fallback.

12 Validation, reproducibility, and what remains untested

12.1 Executed exact reference checks

The archive includes a separate Python implementation using exact SymPy coefficient domains. The recorded run used Python 3.13.5 and SymPy 1.14.0 with deterministic seed 206618. It completed 574 assertions, all passing, in approximately 5.654 seconds in the preparation environment. This is an environment-specific check duration, not a Wolfram performance claim. The individual results are in `validation/reference_results.json`.

The tests include the original radical example; the nested degree-24 example; monomial enumeration counts and cap boundaries; both Chebyshev chains; composite-degree indecomposable examples; constants and linear polynomials; 80 randomized constructed pairs over $\mathbb{Q}$; 45 over $\mathbb{Q}(\sqrt{2})$; a quintic number field and 12 additional randomized pairs there; and the mixed complex degree-eight field.

Several checks deliberately use operations independent of the decomposition code. Forty polynomial divisions are compared with SymPy's division algorithm. Twenty candidate computations are compared with a formal binomial-series expansion. Random constructed examples supply known normalized right and outer factors through (2), rather than asking another decomposition routine to invent an expected answer. Exact recomposition is checked throughout.

These checks support the mathematical recurrence, indexing, polynomial arithmetic, recursive behavior, and expected results. They do not execute Wolfram pattern matching, option dispatch, `RootReduce`, or the native input-domain checks.

12.2 A useful warning from the external oracle

The installed SymPy 1.14.0 decomposition routine was also tried as a secondary comparator. It missed some known decompositions with explicit field domains. A minimal observed example was

```
import sympy as s
x = s.symbols('x')
p = (x**3 + x**2 + x)**2
s.Poly(p, x, domain=s.QQ).decompose()
# Observed in the installed 1.14.0: a singleton degree-six polynomial.
```

The input visibly supplies an outer square and a cubic inner factor. Inspection of that installed implementation found a missing term in its right-candidate recurrence, producing $x^3 + x^2 + 3x/2$ instead of the correct $x^3 + x^2 + x$ over $\mathbb{Q}$. Integer-domain quotient behavior could accidentally hide this particular defect.

Eighteen of 24 selected constructed-composite comparisons disagreed in degree multiset. These observations are retained in the JSON results, not discarded. The archive's `validation/ORACLE_NOTE.md` gives the minimal reproduction, the affected indexing, and the diagnosed update. This is a statement about the installed version that was inspected, not an assertion about all or current SymPy releases. No SymPy patch or upstream submission is included.

Accordingly the external decomposition oracle is not used as the completeness authority for the 574 passing checks. The known constructions, formal-series checks, exact division checks, and the proofs above are independent of that oracle's candidate routine. The observed discrepancy also illustrates why recomposition alone cannot establish completeness: returning the entire input as a singleton always recomposes correctly.

12.3 Prepared native tests, not a native test run

The package includes 107 MUnit `VerificationTest` cases, using exact comparisons. These cover input contracts, original and nested expected chains, algebraic-number representations, selected root conjugates, exact leading cancellation, principal radical semantics, rejection of inexact and transcendental coefficients, unknown options, empty-chain conventions, completeness verification, and enumeration limits. Forty of the cases are frozen randomized rational or quadratic-field compositions; the others include explicit analytic and negative fixtures. Wolfram documents file-based execution through `TestReport` and `VerificationTest` [8, 9].

Native execution was not available. The Wolfram connector was attempted for both context and evaluation and returned HTTP 404. No local Wolfram kernel was available. Therefore the 107 MUnit tests are supplied but unexecuted. They must not be added to the 574 executed reference checks to produce a misleading total of native or executed tests.

Six Wolfram source/test files passed a lexical check for balanced brackets, nested comments, and strings. The same script checks the package core for prohibited reliance on factoring, solving, built-in decomposition, or numerical fallbacks. This is a lexical inspection, not a Wolfram parser or a substitute for native execution.

12.4 Reproducing the checks

From the extracted archive root, run

```
python validation/run_reference_tests.py
python validation/wl_static_check.py
wolframscript -file Tests/RunTests.wls
```

The first two commands reproduce the executed checks. The third is the native test step still to perform in an available kernel. It loads the package, prints the kernel version, runs the test file, and exits nonzero unless the report succeeds.

The runner prefers the modern `ReportSucceeded` report property and includes a fallback using older `success/failure-count` properties. That compatibility code is itself unexecuted. Alternatively, after loading the package in a fresh notebook kernel, run

```
TestReport[FileNameJoin[
  {base, "Tests", "AlgebraicDecomposition.wlt"}]]
```

Use a fresh kernel to avoid definitions attached to symbols in the test fixtures. The archive records native validation as pending rather than claiming a passed platform matrix.

13 Boundaries and further implementations

13.1 Positive characteristic is a genuinely different problem

The triangular uniqueness proof only needs the outer degree m to be invertible, but the convenient differential recurrence also divides by k . In positive characteristic it cannot be transplanted blindly, even in some cases where the triangular argument remains valid. Characteristic zero avoids both issues.

When the outer degree is divisible by the characteristic, same-degree uniqueness itself can fail. In $\mathbb{F}_4$, let a run through the three nonzero elements. Since $a^3 = 1$ and $2 = 0$,

$$\begin{aligned}(x^2 + a^2x) \circ (x^2 + ax) &= (x^2 + ax)^2 + a^2(x^2 + ax) \\ &= x^4 + a^2x^2 + a^2x^2 + a^3x \\ &= x^4 + x.\end{aligned}$$

The three different right components are all monic, zero-constant, and of degree two. This directly disproves a characteristic-free version of Theorem 2.2. The supplied package intentionally has no `Modulus` option.

13.2 Parameters and specialization

The recurrence and monic division can be implemented over a rational function field such as $\mathbb{Q}(u_1, \dots, u_s)$. That would decide generic decomposability in the parameters. Specialization is a separate issue: a leading coefficient may vanish, an intermediate denominator may vanish, or obstruction coefficients may become zero and create a new decomposition.

A parameter-aware extension should return conditions on parameter space, not silently substitute symbolic equality heuristics for exact zero testing. The closed conditions described in Section 6 give a starting point: clear valid denominators on each degree chart, then analyze the simultaneous vanishing of all obstruction coefficients. No such condition solver is included in this release.

13.3 Faster arithmetic and candidate construction

The most direct performance upgrade is an explicit common-number-field backend, with coefficient arithmetic in a fixed quotient field. It can reuse the minimal polynomial and rational basis representation instead of asking RootReduce to canonicalize unrelated intermediate syntax. The decomposition core need not change if the backend preserves exact zero and arithmetic contracts.

For large d , formal root extraction can use precision doubling. With B correct modulo t^k , a Newton correction for $B^m - C$ gives a root modulo approximately t^{2k} :

$$B_{\text{new}} = B - \frac{B^m - C}{mB^{m-1}} \pmod{t^{2k}}.$$

The denominator is a unit of the formal series ring, because its constant term is $m \neq 0$. Fast multiplication, inversion, and truncated powers can then replace the quadratic scalar recurrence. These are potential optimizations, not implemented alternatives selected by a package option.

13.4 Modular screening must respect specialization

A future rational or number-field backend could reduce trial computations modulo suitable primes or prime ideals to reject many degrees cheaply. Such a rejection is valid only when reduction is defined for every coefficient used and the necessary leading coefficients and integer denominators remain nonzero. Bad reduction can destroy a genuine characteristic-zero decomposition or change its degree, so a modular failure without these conditions is not a certificate.

Conversely a successful modular decomposition is only a screening result: an obstruction can vanish modulo one prime without vanishing in characteristic zero. Lift and reconstruct candidates if useful, but retain an exact characteristic-zero recomposition and decision step. The delivered implementation takes this conservative exact route directly.

13.5 Sparse inputs, rational functions, and approximate data

Sparse polynomial data may merit sparse coefficient storage or direct recognition of power and Chebyshev forms. The current dense vectors are not designed to exploit a huge gap between degree and number of terms.

Rational functional decomposition requires handling numerator/denominator cancellation and Möbius rather than merely affine equivalence. The polynomial top-coefficient proof and one-candidate formula are not an API for that different problem. Similarly, a polynomial identity holding only approximately requires a norm, perturbation model, and numerical conditioning analysis. Exact algebraic input is a deliberate boundary of this implementation, not a request to reinterpret decimals as exact nearby numbers.

14 Conclusions

The obstruction to algebraic-coefficient decomposition is not a lack of an algorithm, and it is not necessary to factor the derivative over an ever larger extension. For each candidate right degree, normalization and the leading coefficients determine one possible inner polynomial. The formal root recurrence constructs it inside the original coefficient field. Repeated monic division then provides either its unique outer polynomial or a finite exact obstruction.

These local decisions are enough to obtain one complete chain, all normalized right components, or every normalized complete chain. The same framework handles radicals and selected algebraic Root values without branch guessing. Ritt theory explains the degree invariants and the genuine nonuniqueness, while the implementation's correctness follows from the explicit candidate and membership proofs.

The archive contains the complete Wolfram source, examples, a native test suite ready to run, and an independently executed exact reference implementation. The outstanding validation step is native Wolfram kernel execution; it is recorded as such rather than obscured by the passing reference checks.

A Compact implementation map

Internal routine	Responsibility
inputVector	Validate evaluated input, canonicalize coefficients, remove leading zeros.
red, normalVector	Exact coefficient reduction and vector normalization.
vAdd, vSubtract, vMultiply	Dense exact polynomial-vector arithmetic.
vCompose, vComposeChain	Horner composition and outermost-first chain evaluation.
vDivideMonic	Explicit monic long division.
rightCandidate	Recurrence (6) and zero-constant normalization.
outerData	h -adic membership test and first obstruction digit.
trialData	Combine candidate, membership, and exact pair verification.
rightPairs	Successful normalized pairs over all proper degree divisors.
completeVectorChain	Recursive decomposition of both sides of a successful split.
verifyChain	Exact recomposition identity check.

The full source is `Kernel/AlgebraicDecomposition.wl`. The two core listings in Section 8 are reproduced for explanation; the package file is the authoritative implementation to load. No generated binary, external service, or online connection is needed by the package itself once a Wolfram kernel is available.

B A small manual correctness checklist

For a new implementation of the same algorithm, the following invariants are particularly effective at detecting subtle mistakes:

1. The candidate is monic and zero-constant, with degree exactly d . Its top d coefficients make $p - a_n h^m$ have degree at most $n - d$.
2. The recurrence has the coefficient $(m + 1)j - mk$, not $mj - mk$ or another expression missing the j contribution from $C'B$.
3. Monic division satisfies $p = qh + r$ exactly and $\deg r < d$. The zero polynomial has a consistent representation.

4. Success means *all* h -adic digits are constant, not merely the first one. A first constant remainder does not prove composition.
5. Both sides of a successful split are processed when constructing a complete chain, unless an explicit proof guarantees one side is already indecomposable.
6. Enumerating all binary parenthesizations is not the same as enumerating distinct chains. Use the rightmost indecomposable component or an equally rigorous uniqueness strategy.
7. A cap reached exactly at the last answer is not truncation. Look ahead for another distinct chain before setting an incomplete flag.
8. Exact recomposition establishes equality, but not indecomposability. Negative degree tests or known prime degrees are additionally required.

C Archive layout and rebuilding the article

```
AlgebraicDecomposition/
  README.md
  LICENSE.txt
  PacletInfo.wl
  Kernel/
    AlgebraicDecomposition.wl
    init.m
  Examples/Examples.wl
  Tests/
    AlgebraicDecomposition.wlt
    RunTests.wls
  Reference/exact_decomposition.py
  article/
    article.tex
    article.pdf
    build.sh
  validation/
    STATUS.md
    ORACLE_NOTE.md
    run_reference_tests.py
    reference_results.json
    worked_examples.txt
    make_native_tests.py
    native_test_count.txt
    wl_static_check.py
    wl_static_results.json
```

The TeX source is self-contained and uses a traditional bibliography; no external bibliography processor is required. With a suitable TeX Live installation, build it from the article directory using

```
latexmk -pdf -interaction=nonstopmode -halt-on-error article.tex
```

The preamble names the required font and layout packages. Their font files are not distributed in the archive. Original code, tests, reference implementation, and exposition are supplied under MIT-0; cited works retain their own rights.

References

- [1] Vladimir Reshetnikov, *Is it possible to make Decompose work with coefficients containing radicals?* Mathematica Stack Exchange, question 206618, with answer 206622 and discussion, 2019. The question, answer code, and Daniel Lichtblau’s historical comment were also supplied in the task materials. <https://mathematica.stackexchange.com/q/206618/7288>; <https://mathematica.stackexchange.com/a/206622/7288>.
- [2] Dexter Kozen and Susan Landau, *Polynomial decomposition algorithms*, Journal of Symbolic Computation 7 (1989), 445–456. Bibliographic data and author abstract available on Dexter Kozen’s publication page: https://www.cs.cornell.edu/~kozen/Papers/papers_by_year.htm. The present article derives its working recurrence and correctness arguments explicitly rather than relying on unquoted implementation details from this reference.
- [3] Michael E. Zieve and Peter Müller, *On Ritt’s polynomial decomposition theorems*, preprint dated 23 July 2009. In particular, Section 2, Corollary 2.12, Theorem 2.17, and Section 5.2. <https://websites.umich.edu/~zieve/papers/peter.pdf>.
- [4] Wolfram Research, *Decompose*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/Decompose.html>.
- [5] Wolfram Research, *RootReduce*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/RootReduce.html>.
- [6] Wolfram Research, *ToNumberField*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/ToNumberField.html>.
- [7] Wolfram Research, *Algebraics*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/Algebraics.html>.
- [8] Wolfram Research, *TestReport* and *TestReportObject*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/TestReport.html>; <https://reference.wolfram.com/language/ref/TestReportObject.html>.
- [9] Wolfram Research, *VerificationTest*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/VerificationTest.html>.

Web references were consulted on 7 September 2026. External publications and documentation are cited, not redistributed.